

Билет 14. Реализация `std::future` и `std::promise`. Неудобства `std::future`. `Subscribe`, `ThenValue`, `ThenError`, `Collect*`

0. Формулировка билета

Реализация классов `std::future` и `std::promise`. В чём их неудобство?

Главная идея:

`std::future` и `std::promise` — это две стороны одного общего состояния. `promise` устанавливает результат, а `future` получает его. Сам результат хранится не в `future` и не в `promise`, а в отдельном `shared state`.

Упрощённо:

```
promise —> shared state <— future
        ready flag
        value / exception
        mutex
        condition_variable
```

`promise`:

```
set_value / set_exception
```

`future`:

```
wait / get
```

1. Зачем вообще нужны `future` и `promise`

Представим, что мы хотим запустить задачу в другом потоке и получить результат позже.

Наивно:

```
int result;

std::thread t([&] {
```

```
        result = compute();
    });

    t.join();

    std::cout << result << "\n";
```

Проблемы:

непонятно, когда result готов
нужна синхронизация
нужно передавать исключения
нужно корректно пережить ситуацию, когда producer умер

`future/promise` решают это:

```
std::promise<int> p;
std::future<int> f = p.get_future();

std::thread t([p = std::move(p)]() mutable {
    try {
        int result = compute();
        p.set_value(result);
    } catch (...) {
        p.set_exception(std::current_exception());
    }
});

int value = f.get();
```

Смысл:

worker считает
promise кладёт результат
future ждёт и забирает результат

2. `shared state`

Главный объект реализации — **shared state**.

Это объект в куче, на который ссылаются и `future`, и `promise`.

Упрощённо:

```
template <typename T>
struct SharedState {
    std::mutex m;
    std::condition_variable cv;

    bool ready = false;
    bool has_value = false;

    std::optional<T> value;
    std::exception_ptr exception;
};
```

Почему он нужен?

Потому что `future` и `promise` могут жить в разных потоках и в разное время.

Например:

```
thread #1 владеет future
thread #2 владеет promise
```

Они должны общаться через общий объект.

3. Состояния shared state

Упрощённо shared state может быть в таких состояниях:

```
not ready:
    результата ещё нет

ready with value:
    producer положил значение

ready with exception:
    producer положил исключение

broken promise:
    promise умер, не положив ни value, ни exception
```

Схема:

```
not ready
|
└─ set_value    → ready with value
```

```
|— set_exception → ready with exception
|— promise dtor  → ready with broken_promise
```

Важно:

`future` ждёт не сам `promise`, а готовность shared state.

4. Реализация `promise::set_value`

`promise::set_value` должен:

проверить, что `promise` валиден
проверить, что результат ещё не установлен
положить значение
поставить `ready = true`
разбудить ожидающих

Упрощённый код:

```
template <typename T>
class Promise {
public:
    explicit Promise(std::shared_ptr<SharedState<T>> state)
        : state_(std::move(state))
    {}

    void set_value(T value) {
        if (!state_) {
            throw PromiseInvalid();
        }

        {
            std::lock_guard<std::mutex> lock(state_->m);

            if (state_->ready) {
                throw PromiseAlreadySatisfied();
            }

            state_->value = std::move(value);
            state_->has_value = true;
            state_->ready = true;
        }

        state_->cv.notify_all();
    }

private:
```

```
std::shared_ptr<SharedState<T>> state_;  
};
```

Почему `notify_all`, а не `notify_one`?

Потому что в общем случае ждать могут разные сущности:

```
wait()  
get()  
shared_future consumers  
внутренние continuations
```

Для обычного `std::future` потребитель обычно один, но реализация через `notify_all` проще и безопаснее.

5. Реализация `promise::set_exception`

Исключение тоже является результатом асинхронной операции.

```
void set_exception(std::exception_ptr e) {  
    if (!state_) {  
        throw PromiseInvalid();  
    }  
  
    {  
        std::lock_guard<std::mutex> lock(state_->m);  
  
        if (state_>ready) {  
            throw PromiseAlreadySatisfied();  
        }  
  
        state_>exception = std::move(e);  
        state_>ready = true;  
    }  
  
    state_>cv.notify_all();  
}
```

Потом `future.get()` не просто вернёт значение, а перебросит исключение.

Смысл:

```
исключение из worker-потока переносится в поток, который вызывает  
future.get()
```

6. Реализация `future::wait`

`future::wait` должен дожидаться, пока shared state станет ready.

```
template <typename T>
class Future {
public:
    Future& wait() {
        if (!state_) {
            throw FutureInvalid();
        }

        std::unique_lock<std::mutex> lock(state_->m);

        state_->cv.wait(lock, [&] {
            return state_->ready;
        });

        return *this;
    }

private:
    std::shared_ptr<SharedState<T>> state_;
};
```

Почему через `wait(lock, pred)`?

Потому что это стандартный паттерн condition variable:

```
while (!ready) {
    cv.wait(lock);
}
```

Он защищает от:

```
spurious wakeup
потерянного пробуждения
раннего notify
```

7. Реализация `future::get`

`get` делает две вещи:

ждёт готовности
возвращает value или перебрасывает exception

Упрощённо:

```
T get() {  
    wait();  
  
    std::lock_guard<std::mutex> lock(state_->m);  
  
    if (state_->exception) {  
        std::rethrow_exception(state_->exception);  
    }  
  
    return std::move(*state_->value);  
}
```

Для `std::future` важно:

`get()` обычно можно вызвать только один раз.

Почему?

Потому что значение может быть move-only.

Например:

```
std::future<std::unique_ptr<int>>> f;
```

Когда мы сделали:

```
auto ptr = f.get();
```

значение перемещено наружу. Второй раз его уже нет.

Для многократного чтения существует:

```
std::shared_future<T>
```

8. `future` и `promise` как move-only handles

Обычно `future` и `promise` — это не сами данные, а handle на shared state.

```
Future<T>:  
    shared_ptr / pointer на SharedState<T>  
  
Promise<T>:  
    shared_ptr / pointer на тот же SharedState<T>
```

Но сами объекты логически move-only.

Почему?

у promise должен быть один producer
у обычного future должен быть один consumer

Если переместили `future`:

```
auto f2 = std::move(f);
```

то старый `f` становится invalid.

Попытка пользоваться старым future должна давать ошибку:

FutureInvalid

То же самое с `promise`.

9. PromiseAlreadySatisfied

`promise` можно удовлетворить только один раз.

```
p.set_value(1);  
p.set_value(2); // ошибка
```

или:

```
p.set_exception(...);  
p.set_value(1); // ошибка
```

Почему?

Потому что shared state — one-shot.

Он переходит:


```
not ready → ready
```

и дальше уже не меняется.

Ошибка:

```
PromiseAlreadySatisfied
```

10. Broken promise

Важный случай:

```
std::future<int> f;

{
    std::promise<int> p;
    f = p.get_future();
} // promise уничтожился, ничего не установив

f.get(); // что делать?
```

Если `promise` уничтожился, а результата нет, `future` не должен ждать вечно.

Поэтому деструктор `promise` обычно делает:

```
если shared state ещё не ready:
    положить broken_promise exception
    ready = true
    notify_all
```

Упрощённо:

```
~Promise() {
    if (!state_) {
        return;
    }

    bool should_notify = false;

    {
        std::lock_guard<std::mutex> lock(state_>m);

        if (!state_>ready) {
```

```

        state_->exception =
            std::make_exception_ptr(PromiseBroken());
        state_->ready = true;
        should_notify = true;
    }
}

if (should_notify) {
    state_->cv.notify_all();
}
}

```

Смысл:

Если producer исчез, consumer получает ошибку, а не вечное ожидание.

11. `std::packaged_task`

`std::packaged_task` — удобная обёртка для задачи, которая должна вернуть результат через future.

Например, thread pool обычно принимает `void()` задачи.

Но мы хотим:

```

auto f = pool.submit([] {
    return compute();
});

```

Решение:

```

template <typename F>
auto submit(ThreadPool& pool, F f) -> std::future<decltype(f())> {
    using R = decltype(f());

    auto task =
        std::make_shared<std::packaged_task<R()>>(std::move(f));

    std::future<R> future = task->get_future();

    pool.submit([task] {
        (*task)();
    });

    return future;
}

```

Что происходит:

```
packaged_task хранит callable  
get_future возвращает future на результат  
worker вызывает packaged_task::operator()  
результат или exception попадает в shared state
```

То есть `packaged_task` — это связка:

```
callable + promise-like shared state
```

12. Чем неудобен `std::future`

`std::future` решает базовую задачу:

```
запустил работу  
потом дождался результата
```

Но для асинхронного сервера этого мало.

Главные неудобства:

```
get/wait блокируют поток  
нет Subscribe  
нет then/continuations  
нет удобного chaining  
нет combining  
нельзя задать executor  
есть динамическая аллокация shared state  
обычный future — один потребитель  
у std::async есть ловушка с блокирующим деструктором
```

Разберём подробнее.

13. Проблема 1. Blocking wait/get

`future.get()` блокирует поток.

В обычном коде это нормально:

```
auto result = f.get();
```

Но в event loop это плохо.

Например:

```
I/O-thread:  
  epoll_wait  
  handle client  
  future.get()
```

Если future ещё не готова:

```
I/O-thread заснул  
epoll_wait больше не вызывается  
клиенты не обслуживаются
```

То есть для event-driven сервера нужен не blocking get, а неблокирующая подписка:

```
future.Subscribe(callback);
```

14. Проблема 2. Нет композиции

`std::future` плохо выражает цепочку:

```
A → B → C
```

Без continuations получается:

```
auto a = AsyncA().get();  
auto b = AsyncB(a).get();  
auto c = AsyncC(b).get();
```

Это блокирующий код.

Или callbacks:

```
AsyncA([=](A a) {  
  AsyncB(a, [=](B b) {  
    AsyncC(b, [=](C c) {  
      process(c);  
    }  
  }  
})
```

```
    });  
  });  
});
```

Это callback hell.

Хочется:

```
AsyncA()  
  .ThenValue(AsyncB)  
  .ThenValue(AsyncC)  
  .ThenValue(process);
```

15. Проблема 3. Нет combining

Часто нужно не последовательное:

A → B → C

а параллельное:

A и B и C одновременно

Например:

сходить в три сервиса
дождаться всех
или взять первый успешный ответ
или поставить timeout

У `std::future` из коробки нет удобного:

```
CollectAll  
CollectAny  
CollectAnyWithoutException
```

Приходится писать общий контекст вручную:

результаты
mutex

```
remaining counter  
callback
```

16. Проблема 4. Нельзя задать, где выполнится continuation

Даже если добавить `Subscribe`, появляется вопрос:

в каком потоке выполнится `callback`?

Есть два сценария.

Сценарий 1. Результат уже готов

```
producer уже сделал set_value  
consumer вызывает Subscribe(callback)  
callback выполнится прямо в потоке consumer-a
```

Сценарий 2. Callback подписался раньше

```
consumer вызывает Subscribe(callback)  
producer позже делает set_value  
callback выполнится в потоке producer-a
```

То есть поток исполнения недетерминирован.

Это опасно:

```
тяжёлый callback может случайно выполниться в I/O-thread  
I/O-операция может случайно выполниться в CPU-worker-e  
возникают гонки за состояние connection
```

Решение:

```
executor
```

То есть continuation надо не просто вызвать, а отправить в нужное место:

```
executor->Execute(callback);
```

17. `Subscribe`: зачем он нужен

`Subscribe` — это неблокирующий способ сказать:

когда результат будет готов,
выполни `callback`

Вместо:

```
auto x = future.get(); // блокирует  
process(x);
```

хочется:

```
std::move(future).Subscribe([](Result<T>&& result) noexcept {  
    process(std::move(result));  
});
```

`Result<T>` хранит:

либо значение `T`
либо `exception_ptr`

Это удобно, потому что `callback` получает не только успешный результат, но и ошибку.

18. Сложность реализации `Subscribe`

У нас есть две стороны:

consumer вызывает `Subscribe(callback)`
producer вызывает `SetValue` / `SetError`

Они могут прийти в любом порядке.

Вариант 1. Сначала `Subscribe`

`Subscribe(callback)`
результата ещё нет
`callback` сохраняется

`SetValue(value)`

видит, что callback уже есть
вызывает callback(value)

Вариант 2. Сначала SetValue

SetValue(value)
callback ещё нет
value сохраняется

Subscribe(callback)
видит, что result уже есть
сразу вызывает callback(value)

Нужно корректно обработать оба порядка.

19. Автомат состояний для Subscribe

Можно завести атомарный тег состояния:

```
enum class State {  
    Empty,  
    OnlyResult,  
    OnlyCallback,  
    Done  
};
```

Смысл:

Empty:
нет ни результата, ни callback

OnlyResult:
результат есть, callback ещё нет

OnlyCallback:
callback есть, результат ещё нет

Done:
результат и callback встретились, callback вызван

Переходы:

Subscribe first:
Empty → OnlyCallback


```
SetValue later:
    OnlyCallback → Done
    call callback(result)
```

```
SetValue first:
    Empty → OnlyResult
```

```
Subscribe later:
    OnlyResult → Done
    call callback(result)
```

Ключевая идея:

Кто пришёл первым — сохраняет свою часть. Кто пришёл вторым — видит, что вторая часть уже есть, и вызывает callback.

20. Почему в `Subscribe` легко получить data race

Наивно можно написать:

```
void Subscribe(Callback cb) {
    if (state == Empty) {
        callback = std::move(cb);
        state = OnlyCallback;
    } else {
        callback(result);
    }
}

void SetValue(T value) {
    if (state == Empty) {
        result = std::move(value);
        state = OnlyResult;
    } else {
        callback(result);
    }
}
```

Проблема:

state, callback и result читаются и пишутся разными потоками
если нет синхронизации — data race

Даже если `state` атомарный, важно:

записать callback до публикации состояния OnlyCallback
записать result до публикации состояния OnlyResult
увидеть result/callback после чтения состояния другой стороны

То есть нужны корректные atomic operations и memory order.

Упрощённо для экзамена можно сказать:

CAS по атомарному тегу решает, кто пришёл первым, а release/acquire гарантируют видимость записанных callback/result.

21. Упрощённый `Subscribe` через CAS

Пусть shared state:

```
template <typename T>
struct SharedState {
    std::atomic<State> state{State::Empty};

    Result<T> result;
    Callback<T> callback;
};
```

`Subscribe`:

```
void Subscribe(Callback<T> cb) {
    state_>callback = std::move(cb);

    State expected = State::Empty;

    if (state_>state.compare_exchange_strong(
        expected,
        State::OnlyCallback,
        std::memory_order_release,
        std::memory_order_acquire)) {
        return;
    }

    // Если CAS не удался, значит результат уже есть.
    // В учебной модели expected == OnlyResult.
    state_>callback(std::move(state_>result));

    state_>state.store(State::Done, std::memory_order_release);
}
```

`SetResult`:

```

void SetResult(Result<T> result) {
    state_>result = std::move(result);

    State expected = State::Empty;

    if (state_>state.compare_exchange_strong(
        expected,
        State::OnlyResult,
        std::memory_order_release,
        std::memory_order_acquire)) {
        return;
    }

    // Если CAS не удался, значит callback уже есть.
    // В учебной модели expected == OnlyCallback.
    state_>callback(std::move(state_>result));

    state_>state.store(State::Done, std::memory_order_release);
}

```

Это учебный код. В реальной реализации надо аккуратно обработать:

```

повторный Subscribe
повторный SetValue
исключения
lifetime callback/result
invalid future/promise
одновременное уничтожение объектов

```

22. ThenValue : зачем нужен

`Subscribe` решает проблему блокировки, но им неудобно писать цепочки.

Через `Subscribe`:

```

FooFuture(input).Subscribe([](Result<A>&& ra) {
    if (ra.has_value()) {
        AsyncB(std::move(*ra)).Subscribe([](Result<B>&& rb) {
            if (rb.has_value()) {
                AsyncC(std::move(*rb)).Subscribe([](Result<C>&& rc) {
                    // ...
                });
            }
        });
    }
});

```

```
    }  
});
```

Это снова callback hell.

Поэтому вводят:

```
future.ThenValue(func)
```

Смысл:

если исходная future завершилась значением,
применить func к этому значению
и вернуть новую future

23. Реализация `ThenValue` для обычной функции `T -> U`

Допустим:

```
U func(T value);
```

Тогда:

```
Future<U> g = std::move(f).ThenValue(func);
```

Как это реализовать?

1. Создаём новую пару `Future<U>` / `Promise<U>`.
2. Подписываемся на исходную `Future<T>`.
3. Когда исходная future готова:
 - если там exception:
 - прокидываем exception в `Promise<U>`
 - если там value:
 - вызываем `func(value)`
 - результат кладем в `Promise<U>`
4. Возвращаем `Future<U>`.

Псевдокод:

```
template <typename T>  
template <typename F>  
auto Future<T>::ThenValue(F func) {
```

```

using U = std::invoke_result_t<F, T>;

auto [next_future, next_promise] = GetTied<U>();

std::move(*this).Subscribe(
    [func = std::move(func),
     p = std::move(next_promise)](Result<T>&& result) mutable noexcept {
        try {
            if (result.has_error()) {
                std::move(p).SetError(result.error());
                return;
            }

            U u = func(std::move(*result));
            std::move(p).SetValue(std::move(u));
        } catch (...) {
            std::move(p).SetError(std::current_exception());
        }
    });

return next_future;
}

```

То есть `ThenValue` — это не магия.

Это `Subscribe`, который внутри создаёт новую promise и заполняет её результатом continuation.

24. Проброс исключений в `ThenValue`

Если исходная future завершилась ошибкой:

`ThenValue` не должен вызывать `func`

Он должен просто пробросить ошибку дальше:

```

Future<T> with error
  ↓ ThenValue
Future<U> with same error

```

Схема:

```

f: error("oops")
g = f.ThenValue(func)
g тоже error("oops")

```

Если сама функция `func` бросила исключение:

исключение ловится
кладётся в `next_promise` через `SetError`

Схема:

```
f: value  
func(value) throws "inner"  
g: error("inner")
```

25. `ThenValue` для функции `T -> Future<U>`

Иногда continuation сама асинхронная.

Например:

```
Future<User> GetUserAsync(UserId id);
```

Тогда:

```
auto f = GetUserAsync()  
    .ThenValue([](UserId id) {  
        return GetUserAsync(id);  
    });
```

Функция возвращает:

```
Future<U>
```

Если сделать наивно, получится:

```
Future<Future<U>>
```

Но хочется:

```
Future<U>
```

Поэтому `async-ThenValue` делает `flatten`.

26. Реализация `async ThenValue`

Схема:

1. Создать `next Future<U> / Promise<U>`.
2. Подписаться на исходную `Future<T>`.
3. Когда исходная `Future<T>` успешна:
 - вызвать `func(T)`, получить `inner Future<U>`
 - подписаться на `inner Future<U>`
 - когда `inner` готова:
 - переложить её `Result<U>` в `next Promise<U>`
4. Если исходная `Future<T>` с ошибкой:
 - сразу пробросить ошибку в `next Promise<U>`
5. Если `func` бросила исключение:
 - положить это исключение в `next Promise<U>`
6. Если `func` вернула `invalid Future`:
 - положить `FutureInvalid` в `next Promise<U>`

Псевдокод:

```
template <typename T>
template <typename F>
auto Future<T>::ThenValue(F func) {
    using InnerFuture = std::invoke_result_t<F, T>;
    using U = typename InnerFuture::ValueType;

    auto [next_future, next_promise] = GetTied<U>();

    std::move(*this).Subscribe(
        [func = std::move(func),
         p = std::move(next_promise)](Result<T>&& result) mutable noexcept {
            try {
                if (result.has_error()) {
                    std::move(p).SetError(result.error());
                    return;
                }

                Future<U> inner = func(std::move(*result));

                if (!inner.IsValid()) {
                    std::move(p).SetError(
                        std::make_exception_ptr(FutureInvalid())
                    );
                    return;
                }

                std::move(inner).Subscribe(
                    [p = std::move(p)](Result<U>&& inner_result) mutable
noexcept {
```

```

        std::move(p).SetResult(std::move(inner_result));
    });

    } catch (...) {
        std::move(p).SetError(std::current_exception());
    }
});

return next_future;
}

```

Главная идея:

```

ThenValue(T -> Future<U>)
    подписывается дважды:
        сначала на outer future
        потом на inner future

```

27. ThenError

`ThenError` — это продолжение для ошибочного пути.

Смысл:

```

если future завершилась ошибкой,
вызвать обработчик ошибки

```

Пример:

```

ReadFileAsync()
    .ThenValue(parse)
    .ThenError([](std::exception_ptr e) {
        return default_value();
    });

```

Логика:

```

если исходная future успешна:
    ThenError не вызывает handler
    просто пробрасывает value дальше

если исходная future с ошибкой:

```


вызывает `handler(error)`
его результат становится результатом новой `future`

28. Реализация `ThenError`

Схема похожа на `ThenValue`, только наоборот.

`ThenValue`:
 `value` → обработать
 `error` → пробросить

`ThenError`:
 `value` → пробросить
 `error` → обработать

Псевдокод:

```
template <typename T>
template <typename F>
auto Future<T>::ThenError(F handler) {
    auto [next_future, next_promise] = GetTied<T>();

    std::move(*this).Subscribe(
        [handler = std::move(handler),
         p = std::move(next_promise)](Result<T>&& result) mutable noexcept {
            try {
                if (result.has_value()) {
                    std::move(p).SetValue(std::move(*result));
                    return;
                }

                T recovered = handler(result.error());
                std::move(p).SetValue(std::move(recovered));
            } catch (...) {
                std::move(p).SetError(std::current_exception());
            }
        });

    return next_future;
}
```

Для аsync-варианта:

```
handler(exception_ptr) -> Future<T>
```

нужно сделать flatten так же, как в async `ThenValue` .

29. `CollectAll`

`CollectAll` нужен для параллельной композиции.

```
auto f = CollectAll(std::move(f1), std::move(f2), std::move(f3));
```

Смысл:

дождаться завершения всех future
собрать `Result` каждой future
вернуть `Future<tuple<Result<T1>, Result<T2>, Result<T3>>>`

Важно:

`CollectAll` не падает на первой ошибке. Он ждёт всех и сохраняет ошибки как `Result`.

Схема:

```
f1: value  
f2: error  
f3: value  
  
CollectAll(f1, f2, f3)  
  ↓  
ready после завершения всех  
tuple<Result<T1>, Result<T2>, Result<T3>>
```

30. Реализация `CollectAll`

Нужен общий контекст:

```
template <typename... Ts>  
struct CollectAllContext {  
    std::tuple<Result<Ts>...> results;  
    std::atomic<size_t> remaining;  
    Promise<std::tuple<Result<Ts>...>> promise;  
};
```

Алгоритм:

1. Создаём output Future / Promise.
2. Создаём Context с remaining = N.
3. На каждую input future подписываем callback.
4. Callback i:
 - сохраняет Result_i в results[i]
 - уменьшает remaining
 - если remaining стал 0:
 - SetValue(tuple results)
5. Возвращаем output Future.

Псевдокод:

```
template <typename... Futures>
auto CollectAll(Futures&&... futures) {
    using Output = std::tuple<Result<typename Futures::ValueType>...>;

    auto [out_future, out_promise] = GetTied<Output>();

    auto ctx = std::make_shared<Context<Output>>();
    ctx->remaining = sizeof...(Futures);
    ctx->promise = std::move(out_promise);

    // Для каждой future:
    std::move(future_i).Subscribe(
        [ctx](Result<Ti>&& result) mutable noexcept {
            std::get<i>(ctx->results) = std::move(result);

            if (ctx->remaining.fetch_sub(1) == 1) {
                std::move(ctx->promise).SetValue(
                    std::move(ctx->results)
                );
            }
        });

    return out_future;
}
```

Почему нужен shared context?

Потому что callbacks завершатся в разное время и должны иметь общий доступ к:

```
results
remaining
output promise
```

31. Collect

`Collect` похож на `CollectAll`, но отличается обработкой ошибок.

```
CollectAll:
    ждёт всех
    возвращает Result каждого

Collect:
    если все успешны → возвращает tuple values
    если кто-то завершился ошибкой → output future завершается ошибкой
```

Схема:

```
f1: value
f2: error("oops")
f3: ещё не готова

Collect(f1, f2, f3)
↓
может завершиться сразу error("oops")
```

То есть `Collect` удобен, когда нам нужны все успешные значения, а любая ошибка ломает всю операцию.

32. Реализация `Collect`

Контекст похожий:

```
values
remaining
done flag
output promise
```

Алгоритм:

```
callback i:
    если output уже done:
        return

    если result содержит error:
        если мы первые поставили done:
            promise.SetError(error)
        return
```

```
        иначе сохранить value[i]
        если remaining стал 0:
            если мы первые поставили done:
                promise.SetValue(tuple values)
```

Почему нужен `done flag`?

Потому что несколько future могут завершиться ошибками почти одновременно.

```
f2 error
f3 error
```

Только одна из них должна удовлетворить output promise.

Поэтому:

```
if (!ctx->done.exchange(true)) {
    std::move(ctx->promise).SetError(error);
}
```

33. `CollectAny`

`CollectAny` завершает output future при первой завершившейся input future.

Важно:

```
первая завершившаяся может быть успешной
первая завершившаяся может быть ошибочной
```

Смысл:

```
auto f = CollectAny(std::move(f1), std::move(f2), std::move(f3));
```

Результат содержит:

```
какая позиция завершилась
и её Result
```

Например:

```
f2 завершилась error("second") первой
CollectAny готов
результат = index 1 + error("second")
```

После этого завершения других future уже не меняют output future.

34. Реализация `CollectAny`

Нужен общий флаг:

```
std::atomic<bool> done{false};
```

Callback каждой future:

```
если done.exchange(true) == false:
    мы первые
    записать результат
    promise.SetValue(...)
иначе:
    кто-то уже победил
    ничего не делать
```

Псевдокод:

```
std::move(future_i).Subscribe(
    [ctx](Result<Ti>&& result) mutable noexcept {
        if (ctx->done.exchange(true)) {
            return;
        }

        ctx->result = make_indexed_result<i>(std::move(result));
        std::move(ctx->promise).SetValue(std::move(ctx->result));
    });
```

`CollectAny` используется для:

```
race нескольких запросов
timeout через гонку request future и timer future
hedging
```

35. `CollectAnyWithoutException`

`CollectAnyWithoutException` ждёт первую успешную future.

Отличие от `CollectAny`:

`CollectAny`:
первая завершившаяся, даже error

`CollectAnyWithoutException`:
первая успешная

Сценарий:

f2 завершилась error
игнорируем, ждём дальше

f1 завершилась value
output готов со значением f1

f3 потом завершилась error
уже не важно

Если все завершились ошибками:

output future завершается ошибкой

Какой именно ошибкой?

В учебной реализации можно положить последнюю пришедшую ошибку.

36. Реализация `CollectAnyWithoutException`

Нужны:

done flag
remaining counter
last_error
output promise

Callback каждой future:

```

если done уже true:
    return

если result успешный:
    если мы первые сделали done=true:
        promise.SetValue(indexed value)
    return

если result с ошибкой:
    сохранить last_error
    уменьшить remaining
    если remaining стал 0:
        если мы первые сделали done=true:
            promise.SetError(last_error)

```

Псевдокод:

```

std::move(future_i).Subscribe(
    [ctx](Result<Ti>&& result) mutable noexcept {
        if (ctx->done.load()) {
            return;
        }

        if (result.has_value()) {
            if (!ctx->done.exchange(true)) {
                std::move(ctx->promise).SetValue(
                    make_indexed_value<i>(std::move(*result))
                );
            }
            return;
        }

        ctx->last_error = result.error();

        if (ctx->remaining.fetch_sub(1) == 1) {
            if (!ctx->done.exchange(true)) {
                std::move(ctx->promise).SetError(ctx->last_error);
            }
        }
    });

```

В реальной реализации `last_error` тоже нужно защищать от data race.

Можно использовать:

mutex
или хранить errors по индексам
или аккуратный atomic/context design

37. Почему все эти реализации сложные

Выглядит так, будто future — простой объект:

значение появится позже

Но реализация сложная, потому что есть:

гонка producer и consumer
исключения
move-only значения
одноразовость promise
invalid future/promise после move
broken promise
несколько callbacks в комбинаторах
раннее завершение по ошибке
поток исполнения callback
lifetime shared context
memory order

Главная мысль:

Future — это не просто “коробка со значением”. Это синхронизационный примитив.

38. Что сказать коротко

`std::future` и `std::promise` реализуются через общий shared state в куче. В shared state лежат mutex, condition_variable, флаг ready и результат: значение или exception_ptr. `promise::set_value` или `set_exception` кладёт результат, выставляет ready и будит ожидающих. `future::get` ждёт ready через condition_variable, затем возвращает значение или перебрасывает исключение. Если promise уничтожился, не установив результат, future получает broken promise.

Неудобства `std::future::get/wait` блокируют поток, нет `Subscribe`, нет continuations и chaining, нет combining, нельзя задать executor для продолжения, есть динамическая аллокация shared state, обычный future рассчитан на одного потребителя. Поэтому поверх идеи future строят расширенные futures: `Subscribe`, `ThenValue`, `ThenError`, `CollectAll`, `CollectAny` и executors.

`Subscribe` реализуется как гонка двух событий: `producer` устанавливает `result`, `consumer` устанавливает `callback`. `Shared state` имеет четыре состояния: `Empty`, `OnlyResult`, `OnlyCallback`, `Done`. Кто пришёл первым — сохраняет свою часть, кто пришёл вторым — вызывает `callback`. `ThenValue` строится поверх `Subscribe`: создаётся новая `promise`, исходная `future` подписывается, и при успешном результате вызывается `continuation`. Если `continuation` вернула обычное значение, оно кладётся в новую `promise`; если вернула `future`, делается `flatten`. `ThenError` аналогично обрабатывает ошибочный путь.

Комбинаторы для параллельной композиции используют общий `context`. `CollectAll` ждёт все `future` и собирает все `Results`. `Collect` завершает ошибкой при первой ошибке или значением, когда все успешны. `CollectAny` берёт первую завершившуюся `future`, даже если она завершилась ошибкой. `CollectAnyWithoutException` берёт первую успешную, а если все упали — завершает `output future` ошибкой.

39. Мини-проверка

1. Где физически хранится результат `future/promise`?
2. Что лежит в `shared state` у простой реализации `future`?
3. Что делает `promise::set_value`?
4. Что делает `promise::set_exception`?
5. Как работает `future::wait`?
6. Как работает `future::get`?
7. Что такое `broken promise`?
8. Почему `future/promise` обычно `one-shot`?
9. Чем неудобен `std::future` для `event loop`?
10. Зачем нужен `Subscribe`?
11. Какие четыре состояния можно использовать для реализации `Subscribe`?
12. Как из `Subscribe` реализовать `ThenValue`?
13. Чем `sync ThenValue` отличается от `async ThenValue`?
14. Как работает `ThenError`?
15. Чем `CollectAll` отличается от `Collect`?
16. Чем `CollectAny` отличается от `CollectAnyWithoutException`?